LECTURE 14

Program Synthesis and Evolutionary Search

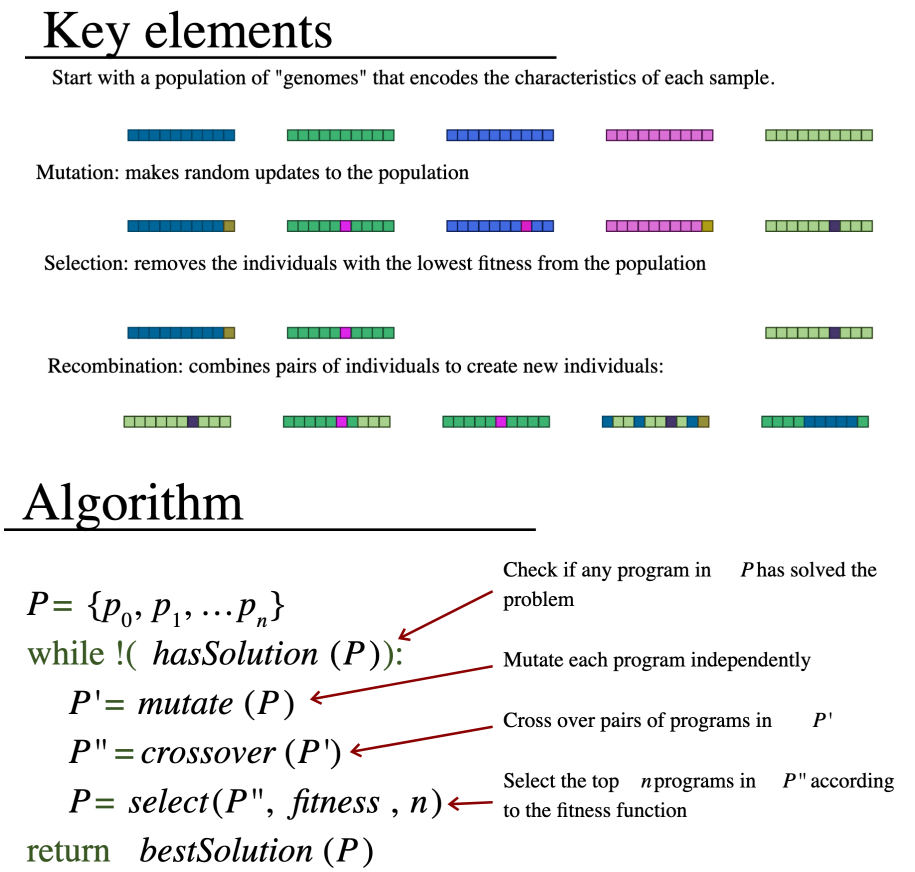
In the past few lectures, we focused primarily on the language models themselves, how to train them, and how to draw samples from them. Despite our best efforts, though, the models may not be very good at solving the specific tasks we care about. For the next couple of lectures, we will now be exploring different approaches that leverage test-time compute to build solutions over multiple invokations of a model.

For this lecture, we focus on the use of evolutionary search in combination with language models to synthesize programs that optimize some real-valued objective function. The objective may be a true quantitative objective, for example, if our program is a game-playing agent, we might want to maximize the score it achieves in the game. But it may also be a smoothed version of a discrete objective; for example, by giving the algorithm partial credit for incorrect solutions that are "close" to the correct solution.

As two examples of this, we will look at Google DeepMind's recent FunSearch and AlphaEvolve projects, which were able to synthesize algorithms that exceeded the best previously known algorithms for important problems such as matrix multiplication and bin-packing.

14.1 A bit of context on Evolutionary algorithms

Evolutionary algorithms first became popular in the 1970s after the publication of John Holland's influential book *Adaptation in Natural and Artificial Systems* [1]. The book introduced the concept of genetic algorithms as a way to solve challenging optimization problems by mimicking the process of natural selection. The idea is to maintain a population of candidate solutions which are evolved through a process of reproduction and selection based on their fitness scores.



The key elements of the evolutionary approach are as follows. First, we need to represent each candidate solution as a *genome*, typically a string of attributes that define the solution. The algorithm will maintain a *population* of such genomes that will be evolved over time. Each genome is subject to *mutation* and *crossover* operations, which introduce variability into the population. Mutation involves randomly changing parts of a genome, while crossover (also called recombination) involves combining parts of two genomes to create a new one.

At every step of the algorithm, it evaluates each candidate solution using a *fitness function*, which assigns a score to each genome based on how well it solves the problem at hand. The algorithm then selects the fittest candidates to form the next generation of the population, while discarding the less fit ones. Each element of the population also undergoes mutation, and surviving candidates are crossed over with each other to create new candidates and replenish the population to its original level.

A particular kind of evolutionary algorithm is *genetic programming*, where the genomes represent computer programs. In this case, the mutation and

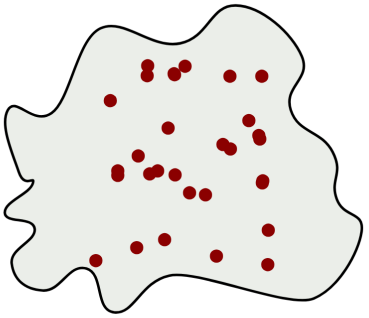
crossover operations are designed to modify the structure of the programs, for example by changing or swapping subtrees in their abstract syntax trees (ASTs). Before the emergence of the modern program synthesis field in the early 2000s, genetic programming was the best known approach to automatic program synthesis.

After significant interest during the 1980s and 1990s, evolutionary algorithms fell out of fashion for some time. Like many other approaches popular during the AI boom of the 1980s, evolutionary algorithms suffered from significant overclaiming in the early days, which inevitably led to disappointment. After their heyday, they were frequently looked down upon by the mainstream machine learning community. Their shortcomings are many. Their main selling point was their generality; they could be applied to any problem whether discrete or continuous, structured or unstructured. But in practice, they rarely worked well without extensive manual effort in the design of the representation, mutation and crossover operators, and fitness function. Unlike some of the sampling-based optimization algorithms we saw in Lecture 15, evolutionary algorithms did not have a strong theoretical foundation that could be used to guide their design. And even when they worked, the solutions could be defficient in other ways. For example, genetic programming often produced 'Frankenprograms' that combined bits and pieces of different programs in ways that were inefficient, difficult to understand, and that may not generalize well beyond the training data.

So why are we talking about evolutionary algorithms now? Despite their limitations, evolutionary algorithms remain a powerful and flexible optimization technique, and recent work has shown that they can be combined with large-language models to reduce some of their shortcomings and produce impressive results.

14.2 FunSearch

FunSearch



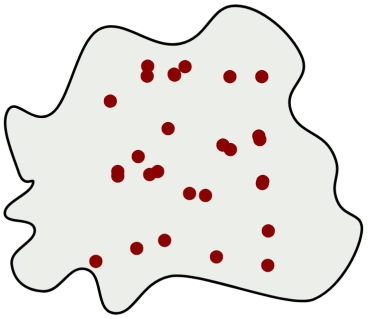
LLM generates initial population of programs based on a prompt

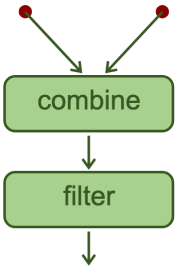
This ensures that while the programs are somewhat random, they are still in distribution for the task

*Romera-Paredes et al., ***Mathematical discoveries from program search with large language models***, Nature volume 625, pages468–475 (2024)

FunSearch

Recombination and mutation are performed as a single step by an LLM that takes multiple programs into its prompt and is asked to provide a new program combining elements of the given programs

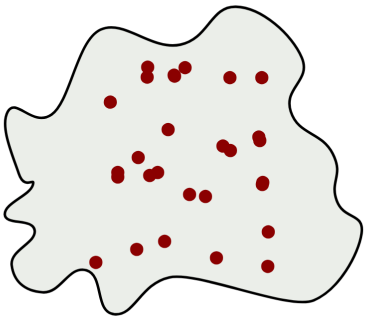


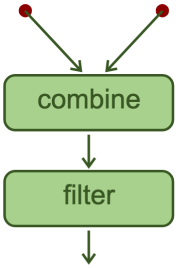


*Romera-Paredes et al., ***Mathematical discoveries from program search with large language models***, Nature volume 625, pages468–475 (2024)

FunSearch

This reduces the issues with Frankenstein programs, because the LLM ensures the combined program is in distribution. It also eliminates the black magic involved in designing good recombination operators.

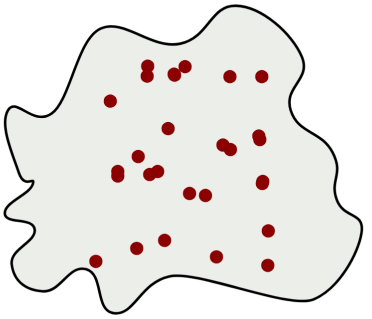


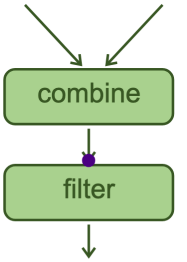


*Romera-Paredes et al., ***Mathematical discoveries from program search with large language models***, Nature volume 625, pages468–475 (2024)

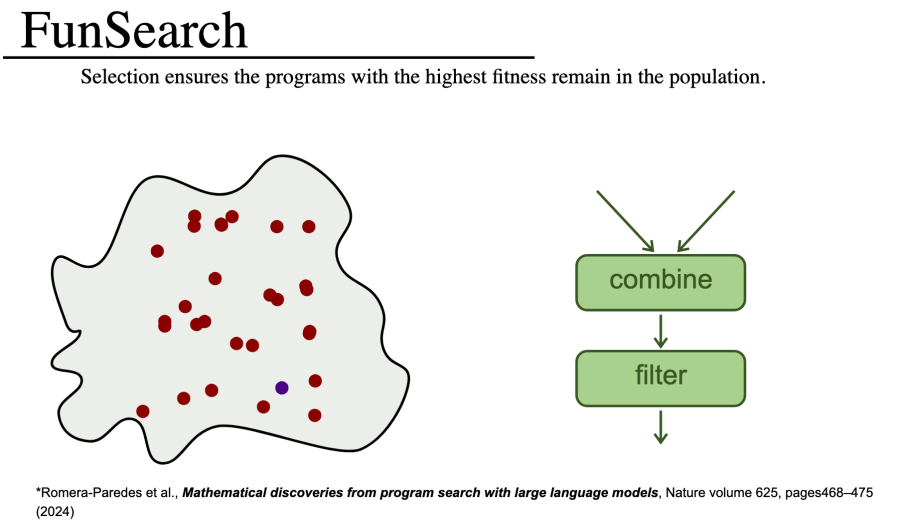
FunSearch

This reduces the issues with Frankenstein programs, because the LLM ensures the combined program is in distribution. It also eliminates the black magic involved in designing good recombination operators.





*Romera-Paredes et al., ***Mathematical discoveries from program search with large language models***, Nature volume 625, pages468–475 (2024)



Back in 2023, Google DeepMind presented a paper that made a startling claim: "a scientific discovery— a new piece of verifiable knowledge about a notorious scientific problem—using an LLM" [2]. Specifically, the paper claimed to have discovered a new lower bound for the size of cap sets in $\mathbb{Z}_3^n$, a problem in combinatorics where the existing lower bounds had stood for 20 years. The specific claim to mathematical significance is controversial, but the approach they used has proven to be quite useful as a program synthesis technique.

The paper presented an algorithm called FunSearch, which combines evolutionary search with large language models to synthesize programs that optimize a given objective function. The key idea is to use a pre-trained language model in place of the traditional mutation and crossover operators. The algorithm also used an LLM to generate the initial population of candidate solutions.

Traditional evolutionary algorithms rely on handcrafted mutation and crossover operators, but they are simply hard to design well. They have the dual goal of introducing enough variability into the population so that the algorithm can get good coverage of the search space, but also have a certain amount of locality so they allow for gradual improvement of candidates over time. Moreover, they can introduce unnatural artifacts into the candidates, from dead code to roundabout ways of implementing simple operations.

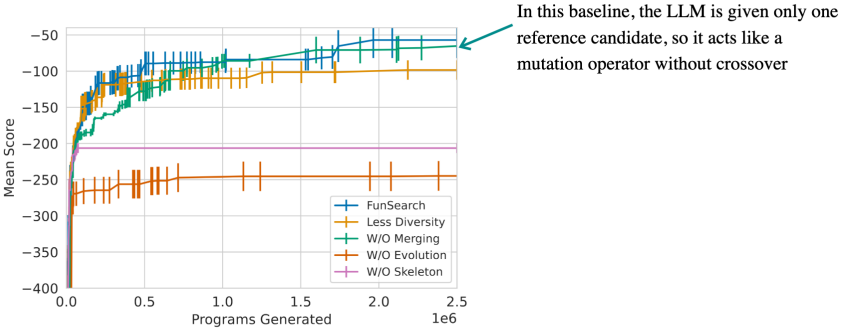
In contrast, by using a large language model to generate mutations and crossovers, FunSearch is able to leverage the knowledge encoded in the model to produce more natural and effective variations of the candidates. This is done by simply providing the LLM with a collection of existing candidates and asking it to generate a candidate that improves on the given examples by combining elements of all of them. The LLM is able to generate code that is syn-

tactically correct and in-distribution for the given domain. In addition to the examples, the LLM is also given the score of each solution, allowing the LLM to take this information into consideration when deciding how to craft a new solution.

This approach is more flexible than traditional handcrafted operators. For example, it is easy to generalize the crossover operation to combine more than two candidates at a time, simply by providing more examples to the LLM. It is also possible to condition the LLM with additional constraints. For example, in the paper they use a sketch of the solution to constrain the search space to programs that fit the sketch.

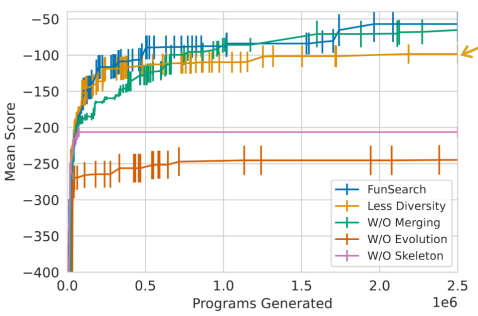
The paper also uses a version of evolutionary search called *island evolutionary algorithm*, where multiple populations (islands) are evolved in parallel, with occasional migration of candidates between islands. This approach helps to improve diversity in the population and prevents premature convergence to suboptimal solutions.

Ablations



From the supplementary material in the original FunSearch (Romera-Paredes et al. 2023) paper.

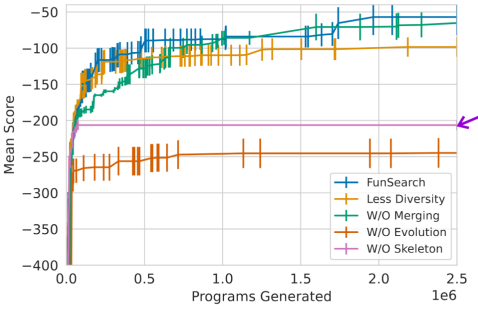
Ablations



This baseline uses a regular GA without islands

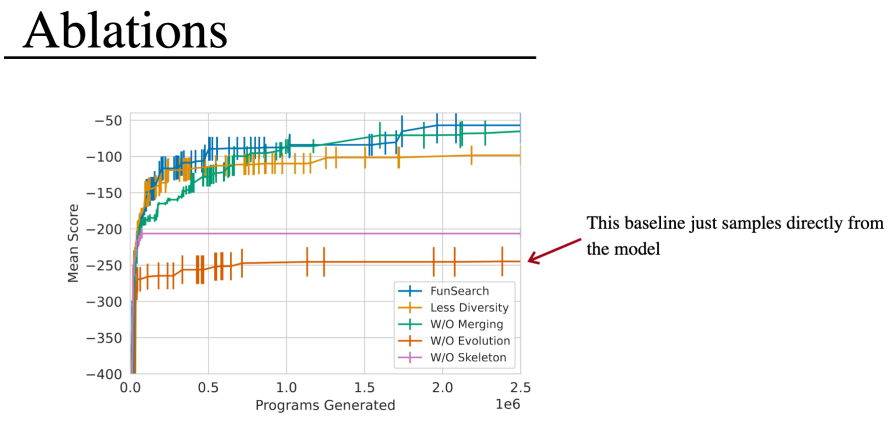
From the supplementary material in the original FunSearch (Romera-Paredes et al. 2023) paper.

Ablations



This baseline doesn't use the skeleton

From the supplementary material in the original FunSearch (Romera-Paredes et al. 2023) paper.



From the supplementary material in the original FunSearch (Romera-Paredes et al. 2023) paper.

The graph in the figure comes from the original paper and it shows the contribution of the different elements of the algorithm to the performance. We can see that the use of multiple parents in the crossover operation actually has a relatively small benefit compared with using only one parent (i.e., mutation only). The use of sketches, on the other hand, has a significant impact on performance. The use of islands also has a small but noticeable benefit. But all the variants of the algorithm significantly improve over the baseline of using only the LLM to generate candidates without any evolutionary search.

14.3 FunSearch for data augmentation

While the original FunSearch paper focused on synthesizing mathematical algorithms, the same approach can be applied to other domains as well, and not just for directly synthesizing a desired function. For example, in a previous lecture, we discussed the VLMaterial project in which we fine tuned a model to generate code to generate procedural textures for 3D models. One of the main challenges in that project was the lack of high-quality training data. The FunSearch approach was used to synthesize new texture generation programs from an initial population of existing programs.

In this approach, the fitness function was a measure of program quality that considered the information content of the generated textures. The idea was to encourage the synthesis of programs that generated textures with high information content; the goal was to avoid programs that generated an interesting texture only to clobber it by, for example, reducing the contrast to zero, or overlaying a solid color on top of it. By optimizing for programs that generated high-information-content textures, the algorithm was able to synthesize

new texture generation programs that produced a diverse range of interesting textures.

The FunSearch approach was well suited for data generation because it was able to leverage the knowledge encoded in the LLM to generate new programs that were syntactically correct and in-distribution for the domain, while still generating a good amount of diversity in the population. Using this approach, we were able to expand an initial set of about 1600 programs up to over half a million, enough to successfully fine tune a model to generate new texture generation programs.

14.4 AlphaEvolve

Very recently, FunSearch was followed up by the AlphaEvolve project [3]. There are two key tecnical differences between FunSearch and AlphaEvolve. First, AlphaEvolve mutates entire program files, rather than just single functions, as was done in FunSearch. This allows for more complex and modular solutions to be synthesized, at the cost of significantly increased computational requirements. The second change is that AlphaEvolve introduced a meta-prompting strategy, where the model also independently searched for and generated *instructions* for how to solve the problem, not just the solutions themselves. These instructions are then stored and sampled from the database, just like the solutions.

Behind the scenes, AlphaEvolve also benefitted from other advances made in language modeling since FunSearch. In particular, the language model itself was much larger and more powerful. From an engineering perspecetive, the authors also built a significant amount of infrastructure to support arbitrary fitness functions (such as, for example, training and evaluating a neural network!), which allowed them to apply the techniques from FunSearch to a much wider range of problems.

The keystone result of AlphaEvolve was that it was able to synthesize a new algorithm for 4x4 matrix multiplication that outperformed the previously best-known algorithm at the time, reducing the number of operations required to multiply two 4x4 matrices from 48 to 49— the previous best-known algorithm had been discovered in 1969.

While FunSearch may have been a proof of concept that large-scale program synthesis could be used to discover new algorithms, DeepMind have signaled that AlphaEvolve is already being applied across several domains of Google to improve real-world systems. An early version of AlphaEvolve has already been used to improve the performance of Google's data center scheduling, recovering 0.7% more stranded resources than could previously be done, by discovering a new scoring function that is used to match jobs to resources in a way that is tailored to Google's computing workloads. The resulting scoring function is

remarkably simple:

```
def alpha_evolve_score(required, free):
    cpu_residual = required.cpu / free.cpu
    memory_residual = required.memory / free.memory
    return -1.0 * (cpu_residual + memory_residual
                   + mem_residual / cpu_residual
                   + cpu_residual / mem_residual)
```

Fitting such scoring functions in an automated data-driven way has been done before with reinforcement learning, but that resulted in black-box scoring functions that were difficult to debug, and frightening to deploy in production. By contrast, treating it a synthesis problem allowed the authors to discover a simple, human-readable scoring function that is significantly easier to trust with real-world resources.

This is an example of the power of combining optimization and search strategies with large language models. In the next lecture, we will explore this further in the context of agents that combine LLM calls with other code to solve complex tasks.

References

[1] John H. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, Ann Arbor, MI, 1975. URL https://direct.mit.edu/books/monograph/2574/Adaptation-in-Natural-and-Artificial-SystemsAn. Reprinted by MIT Press, 1992.

[2] Bernardino Romera-Paredes, Yan Karklin, Guillaume Desjardins, Ethan Dyer, Adria Recasens, Adam Santoro, Aleksandar Botev, Igor Babuschkin, David Reichert, Petar Veličković, et al. Mathematical discoveries from program search with large language models. *Nature*, 623(7987):756–763, 2023. doi: 10.1038/s41586-023-06924-6. URL https://www.nature.com/articles/s41586-023-06924-6. DeepMind: FunSearch system for automated mathematical discovery.

[3] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery. White paper, Google DeepMind, 2025. URL https://storage.googleapis.com/deepmind-media/DeepMind.com/Blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/AlphaEvolve.pdf. © 2025 Google DeepMind. All rights reserved.